

Sistemas Distribuidos

Proyecto final



Alumnos:

Iván González Portero 100522281
Vanesa Elena Ionescu 100522287

Índice de contenidos

Índice de contenidos	1
Descripción del código (Parte 1)	2
Carpeta client	2
Carpeta server/src/c_files & h_files	2
Carpeta server/database	3
Descripción del código (Parte 2)	4
Carpeta cliente	4
Carpeta server/src/c_files & h_files	4
Carpeta server/src/rpc	6
Carpeta server/rpc_server	7
Carpeta server/web_service	8
Compilación del proyecto	9
Parte 1	9
Parte 2	9
Batería de pruebas: Parte 1	10
Batería de pruebas: Parte 2	12
Conclusiones	13

Descripción del código (Parte 1)

En esta primera parte del proyecto, hemos diseñado e implementado la base del servicio de mensajería asíncrona mediante el uso de Sockets TCP. Para el desarrollo del servidor, nos hemos aprovechado de la arquitectura concurrente basada en el patrón maestro-trabajador que ya implementamos en el Ejercicio Evaluable 2.

Carpeta client

client.py

En este fichero se implementa la lógica del cliente. Para cada operación especificada en el enunciado (REGISTER, UNREGISTER, CONNECT, DISCONNECT, SEND, USERS), el cliente crea un socket TCP, se conecta al servidor, serializa los argumentos enviándolos como cadenas de texto plano terminadas en `\0`, y espera el byte de respuesta para imprimir el resultado correspondiente por pantalla.

La parte más compleja de este fichero es la gestión de la operación CONNECT. Al ejecutarse, el programa busca un puerto libre en la máquina local y levanta un hilo secundario (thread) de escucha. Este hilo actúa como un pequeño servidor en escucha constante para poder recibir de forma asíncrona los mensajes que el servidor central le reenvía procedentes de otros usuarios, imprimiéndolos por pantalla sin bloquear el hilo principal de la interfaz de usuario.

Carpeta server/src/c_files & h_files

comunicaciones.c & comunicaciones.h

Estos ficheros están directamente reciclados de nuestro ejercicio de sockets. Contienen las funciones auxiliares `sendMessage` y `recvMessage`, las cuales envuelven las llamadas al sistema `write` y `read` dentro de un bucle `while`. Consideramos que esta es una estrategia estrictamente necesaria y válida para evitar la fragmentación de los paquetes TCP, asegurando que se envían y reciben exactamente la cantidad de bytes requeridos, previniendo lecturas incompletas.

database.c & database.h

En esta parte del proyecto se implementan las funciones encargadas de la persistencia de los datos en disco y el control de las estructuras en memoria, es decir, son estas funciones las que se encargan de manipular los ficheros de la carpeta database.

Aquí se gestionan las lecturas y escrituras de los usuarios registrados y los mensajes pendientes. Para evitar condiciones de carrera cuando varios hilos trabajadores intentan leer o modificar el estado de un usuario simultáneamente, hemos protegido el acceso a estas estructuras mediante el uso de mutexes.

funciones.c & funciones.h

En este fichero se encuentran las funciones que llama la función `tratar_peticion` en el servidor. Contiene las funciones específicas que dan respuesta a cada uno de los comandos del protocolo del enunciado: `procesar_registro`, `procesar_baja`, `procesar_conexion`, etc.

Cada función extrae los argumentos del buffer recibido, interactúa con `database.c` para verificar o alterar el estado de los ficheros de la carpeta, y compone la cadena de respuesta (con su respectivo código de estado 0, 1, 2, o 3) que posteriormente se enviará al cliente.

servidor.c

En este fichero se implementa el hilo principal del servidor, manteniendo exactamente la misma estructura maestro-trabajador del Ejercicio Evaluable 2. El `main` (que sería el maestro) inicializa el socket del servidor, hace el `bind` y el `listen`, y entra en un bucle infinito donde acepta peticiones que le van llegando de los clientes.

Por cada petición entrante, delega el descriptor del socket a un nuevo hilo trabajador que ejecuta la función `tratar_peticion`. Hacemos uso de una variable de condición y un mutex para asegurar que el hilo trabajador ha copiado correctamente el descriptor del socket a su memoria local antes de que el hilo maestro acepte la siguiente conexión. El hilo trabajador se encarga de leer la operación, llamar a la función correspondiente de `funciones.c` para que esta le devuelva la respuesta pertinente al cliente.

Carpeta server/database

usuarios.txt

Es un fichero de texto plano utilizado como base de datos persistente para almacenar a los usuarios que se han registrado exitosamente en el sistema. Aquí se guarda tanto su nombre, como un número que dice si está conectado o no, su IP y el ID del último mensaje.

mensajes_usuario.dat

Es un fichero binario que actúa como buzón temporal. Almacena las tuplas de mensajes que son enviados a usuarios que actualmente se encuentran en estado "Desconectado". Una vez que el usuario destino realiza un `CONNECT`, el servidor lee este fichero, envía los mensajes pendientes y los elimina para liberar espacio.

Descripción del código (Parte 2)

En esta carpeta se encuentra nuestra respuesta a la segunda parte de la práctica. Como los cambios, sobre todo en la carpeta server o en client.py no saltan a la vista de forma sencilla, hemos puesto comentarios específicos (// parte2: comentario) para que sean más fáciles de leer y localizar.

Carpeta cliente

client.py

A las funcionalidades de la parte anterior se le han sumado tres cambios críticos:

- **Integración del Servicio Web:** Antes de enviar cualquier mensaje (SEND o SENDATTACH), el cliente realiza una petición HTTP POST al servicio web local, implementado en la carpeta web_service, enviando el texto original. Recupera el texto normalizado y es ese texto modificado el que viaja al servidor central.
- **Operación SENDATTACH:** Se ha añadido el parseo del nombre del fichero. La lógica hacia el servidor central es casi idéntica a SEND, pero adjuntando el fileName, por lo que se hace un send extra al servidor.
- **Operación GETFILE:** Para obtener un archivo de otro usuario hemos implementado la operación GETFILE. En ella, el cliente primero verifica si tiene almacenada la IP y puerto del usuario remitente (obtenidos previamente al modificar la respuesta de la operación USERS). Si los tiene, establece una conexión TCP directa (socket a socket) con el hilo de escucha del usuario remoto, mandando la orden GET FILE, leyendo el flujo de bytes entrante y escribiéndolo en disco bajo el nombre especificado en <localFileName>.

Carpeta server/src/c_files & h_files

En esta segunda parte, la estructura modular del servidor se ha mantenido, pero los ficheros han sufrido ampliaciones y modificaciones para dar soporte a la transferencia de ficheros, la integración con el servicio RPC y la gestión avanzada de los identificadores.

comunicaciones.c & comunicaciones.h

En estos ficheros no se ha hecho ninguna modificación respecto de la primera parte.

database.c & database.h

Aunque el fichero database.c ya existía en la primera parte, en esta segunda entrega ha necesitado varias modificaciones para dar soporte a las nuevas funcionalidades. El cambio más destacado es la ampliación de la estructura PendingMsg, que ahora incorpora un campo adicional fileName[256]. Gracias a esta adición, la base de datos es capaz de almacenar no solo el cuerpo del mensaje y el remitente, sino también el nombre del fichero

adjunto en aquellas operaciones SENDATTACH cuyos destinatarios se encontrasen desconectados en el momento del envío, de forma que cuando el usuario destino se conecte más adelante, el servidor pueda entregarle el mensaje completo junto con la referencia al fichero que tiene pendiente de descarga.

Por su parte, la función `db_store_message()` ha recibido un nuevo argumento, `const char *fileName`, que escribe este campo en la estructura antes de persistirla en disco mediante `write()`. Cuando dicho argumento llega como `NULL` (caso de un SEND ordinario sin adjunto), simplemente se inicializa el campo a la cadena vacía con un `memset`, lo que garantiza la compatibilidad hacia atrás con el resto del código.

La otra modificación destacada afecta a la función `db_get_connected_users()`. En la primera parte, esta función devolvía únicamente el nombre de cada usuario conectado. En la segunda parte, el enunciado exige que el servidor envíe al cliente, junto con cada nombre, la IP y el puerto de escucha de ese usuario, para que el cliente pueda posteriormente establecer conexiones con otro cliente directamente en la operación GETFILE. Para satisfacer este requisito, `db_get_connected_users()` ahora construye cada entrada de la lista con el formato "nombre :: ip :: puerto" usando `snprintf()`, y es ese string compuesto el que se copia en el array `usersList` que se devuelve al llamador.

funciones.c & funciones.h

Las funciones `f_register()`, `f_unregister()`, `f_connect()`, `f_disconnect()`, `f_users()` y `f_send()` conservan exactamente la misma lógica que tenían en la primera parte. La única diferencia apreciable es la incorporación, al final de cada una de ellas, de una llamada a `log_rpc_send()`, la función definida en `log_rpc_client.c` que actúa como punto de entrada al sistema de registro RPC.

El cambio estructural más relevante de este fichero es la incorporación de la función `f_sendattach()`. Esta función implementa la lógica del nuevo comando SENDATTACH del protocolo y sigue un esquema prácticamente idéntico al de `f_send()`, con la diferencia de que maneja un argumento adicional, `fileName`, que contiene el nombre del fichero adjunto al mensaje.

log_rpc_client.c & log_rpc_client.h

Estos dos ficheros son completamente nuevos en esta segunda parte y constituyen el puente entre el servidor de mensajería y el servidor de registro de operaciones. Su diseño se ha inspirado directamente en el patrón utilizado en el Ejercicio 3 con `proxy-rpc.c`, donde toda la infraestructura RPC quedaba encapsulada detrás de una interfaz limpia y sencilla.

El fichero de cabecera `log_rpc_client.h` expone únicamente tres funciones públicas: `log_rpc_init()`, `log_rpc_destroy()` y `log_rpc_send()`. De esta forma, el resto del servidor (`funciones.c`, `servidor.c`) no necesita conocer nada sobre la infraestructura ONC-RPC para poder registrar operaciones: simplemente llama a `log_rpc_send()` con el nombre de usuario, el nombre de la operación y, en el caso de SENDATTACH, el nombre del fichero adjunto.

La implementación en `log_rpc_client.c` mantiene un puntero `CLIENT*` estático compartido entre todos los hilos del servidor. Este puntero se inicializa una única vez en `log_rpc_init()`, que lee la dirección del servidor RPC de la variable de entorno `LOG_RPC_IP` y establece la conexión usando `clnt_create()` con el protocolo TCP para garantizar fiabilidad en la transmisión. Si la variable de entorno no está definida, la función simplemente desactiva el log de forma silenciosa y retorna -1, pero el servidor de mensajería continúa funcionando con normalidad.

Dado que el servidor es multihilo y este puntero `CLIENT*` está compartido entre todos los hilos trabajadores, el acceso al mismo está protegido por un mutex estático `rpc_mutex`. Así evitamos que dos hilos intenten realizar una llamada RPC de forma simultánea, lo que podría corromper el estado interno del cliente RPC.

Dentro de `log_rpc_send()`, antes de invocar al stub generado por `rpcgen` (`log_operation_1()`), los argumentos se copian a buffers locales de tamaño fijo. Esta precaución es necesaria porque el stub XDR espera recibir punteros a `char*` mutables, y pasar directamente los `const char*` que llegan como argumentos a `log_rpc_send()` podría provocar un comportamiento indefinido en algunas plataformas.

servidor.c

En este fichero, lo más reseñable que hemos modificado para esta entrega ha sido en la función `tratar_peticion()`, donde se ha añadido el tratamiento del nuevo operador `SENDATTACH`. La lógica es la misma que para el resto de operaciones: se leen los argumentos necesarios mediante llamadas sucesivas a `leer_cadena()` y se llama a la función `f_sendattach()` de `funciones.c`. En este caso concreto, son necesarias cuatro lecturas adicionales: el nombre del remitente, el nombre del destinatario, el cuerpo del mensaje y el nombre del fichero adjunto, en ese orden. Una vez procesada la petición, se cierra el descriptor del socket del cliente y se termina la ejecución del hilo.

Por último, en el manejador de señal `manejador_sigint()` se ha incorporado la llamada a `log_rpc_destroy()`, que destruye ordenadamente el cliente RPC antes de cerrar el socket principal del servidor y terminar el proceso. De esta forma se liberan correctamente todos los recursos tanto de red como de memoria asociados a la infraestructura RPC. También hemos añadido al principio de la función `main` la función `log_rpc_init()`.

Carpeta server/src/rpc

log_rpc.x

Esta carpeta contiene la infraestructura RPC del lado del cliente, es decir, los ficheros que utiliza el servidor de mensajería para poder hacer llamadas al servidor de registro. En concreto, incluye el fichero de definición de la interfaz `log_rpc.x` y los tres ficheros generados automáticamente por `rpcgen` que necesita el servidor de mensajería para compilar: `log_rpc.h`, `log_rpc_clnt.c` y `log_rpc_xdr.c`.

El fichero `log_rpc.x` define, en lenguaje IDL/XDR propio de ONC-RPC, el contrato entre el servidor de mensajería y el servidor de registro. En él se declara el tipo cadena como un alias para `string` de hasta 256 caracteres, y se define el único procedimiento remoto del programa: `LOG_OPERATION`, que recibe tres argumentos de tipo cadena (usuario, operacion y fichero) y devuelve un entero como código de resultado.

Hemos definido solo una operación ya que al fin y al cabo, el cometido del registro únicamente es imprimir por pantalla las acciones que hacen los usuarios, acción que salvo en el caso de `SENDATTACH` siempre sigue la misma estructura, por lo que hacer una operación para cada función complicaría la lógica de forma innecesaria.

Carpeta `server/rpc_server`

Esta carpeta es la que contiene el servidor RPC de registro propiamente dicho, es decir, el proceso independiente que hay que arrancar para que el sistema de log funcione.

`log_rpc.x`

Al igual que en la carpeta `src/rpc`, esta carpeta también incluye su propio `log_rpc.x`. Aunque el contenido de ambos ficheros `.x` es idéntico, lo volvemos a poner aquí ya que así podemos compilar el servidor RPC de manera completamente independiente del servidor de mensajería, sin dependencias cruzadas entre ambos proyectos.

`log_rpc_server.c`

Este fichero es donde reside la implementación real de la lógica del servidor RPC, es decir, el código que se ejecuta en el proceso remoto cuando el servidor de mensajería invoca `LOG_OPERATION`. Específicamente, contiene la función `log_operation_1_svc()`, cuya firma es generada automáticamente por `rpcgen` a partir del fichero `.x` y que nosotros tenemos que rellenar.

La implementación que hemos realizado es bastante sencilla, pues el único propósito de este servicio es imprimir por pantalla el registro de la operación. Dentro de `log_operation_1_svc()`, lo primero que se hace es validar que los punteros `usuario` y `operacion` no sean nulos, retornando `-1` en caso de que lo sean. A continuación, se comprueba si el campo `fichero` contiene una cadena no vacía, lo que indica que se trata de una operación `SENDATTACH`. En ese caso, se imprime una línea con el formato `"usuario\toperacion\tfichero"`; en cualquier otro caso, se imprime únicamente `"usuario\toperacion"`.

El resto del fichero, concretamente `log_rpc_1_freeresult()`, se limita a invocar `xdr_free()` para liberar la memoria que `rpcgen` reservó para deserializar los argumentos de entrada, comportamiento estándar en cualquier servidor ONC-RPC.

Carpeta server/web_service

requirements.txt

Este fichero lista las dependencias Python necesarias para ejecutar el servicio web de normalización. Las bibliotecas requeridas son: fastapi, que proporciona el framework web; uvicorn, que actúa como servidor ASGI para servir la aplicación; pydantic, que se encarga de la validación y serialización de los modelos de datos de entrada y salida; y requests, que se utiliza desde el cliente Python para realizar las peticiones HTTP POST al servicio.

ws_normalize.py

Este fichero implementa el servicio web de normalización de texto. Para ello, hemos hecho uso del framework FastAPI junto con el servidor ASGI uvicorn.

El servicio expone un único endpoint relevante para el proyecto: POST /normalize. Este endpoint recibe en el cuerpo de la petición un objeto JSON con el campo message, que contiene el texto a normalizar. Internamente, la lógica de normalización consiste en aplicar la expresión regular `\s+` para sustituir cualquier secuencia de uno o más caracteres de espacio en blanco (espacios, tabuladores, saltos de línea) por un único espacio, y acto seguido eliminar los posibles espacios sobrantes al principio y al final de la cadena mediante `strip()`. El resultado se devuelve en un objeto JSON con el campo normalized. Hemos optado por una expresión regular en lugar de llamadas encadenadas a `replace()` porque resulta más simple y más robusta ante distintos tipos de espaciado que el usuario pudiera introducir desde la terminal.

Adicionalmente, se ha definido un endpoint auxiliar GET / que actúa como health check y devuelve un JSON sencillo indicando que el servicio está operativo. Aunque no forma parte del protocolo con el cliente, resulta útil durante el desarrollo y las pruebas para verificar rápidamente que el servicio está levantado y respondiendo antes de ejecutar el cliente.

El servidor arranca en la dirección 127.0.0.1 y en el puerto 5000, ya que el enunciado especifica que el servicio web se despliega de forma local en la misma máquina que el cliente, y el cliente Python tiene configurada la URL `http://127.0.0.1:5000/normalize` como destino de sus peticiones de normalización.

Compilación del proyecto

Parte 1

Para compilar la Parte 1 del proyecto en su totalidad, únicamente es necesario ejecutar el siguiente comando desde la carpeta parte1/server:

Compilación del servidor: make

Esto generará el ejecutable server en el directorio parte1/server. El cliente, al estar implementado en Python, no requiere compilación. Para lanzar el sistema, en primer lugar debe arrancarse el servidor indicando el puerto en el que escuchará mediante el flag -p:

Ejecución del servidor: ./server -p <Puerto>

A continuación, en otra terminal, se ejecuta el cliente Python indicando la IP del servidor y el puerto con los flags -s y -p respectivamente:

Ejecución del cliente: python3 client.py -s <IP> -p <Puerto>

Parte 2

Para compilar la Parte 2 del proyecto, empezamos con la compilación del servidor de mensajería, que se realiza con el mismo comando que en la primera parte:

Compilación del servidor de mensajería: make

El servidor de registro RPC se compila de forma independiente desde la carpeta parte2/rpc_server:

Compilación del servidor RPC: make

El servicio web de normalización no requiere compilación, pero sí es necesario instalar sus dependencias mediante pip antes de arrancarlo:

Instalación de dependencias: pip install -r requirements.txt

Para la ejecución completa del sistema en la Parte 2 es necesario arrancar los tres procesos en el orden adecuado: primero el servidor RPC de log, después el servicio web de normalización y por último el servidor de mensajería. El servidor de log se arranca simplemente ejecutando su binario, aunque si no se tiene activado el rpc_bind habrá que hacerlo con el siguiente comando: sudo systemctl start rpcbind

Una vez hecho esto ejecutamos el binario:

Servidor RPC: ./log_rpc_server

El servicio web se arranca desde la carpeta parte2/web_service:

Servicio web: python3 ws_normalize.py

Finalmente, el servidor de mensajería se lanza con la variable de entorno LOG_RPC_IP apuntando a la máquina donde corre el servidor de log (normalmente localhost en pruebas locales):

Servidor de mensajería: LOG_RPC_IP=localhost ./server -p <Puerto>

El cliente Python de la Parte 2 se ejecuta de la misma forma que en la Parte 1:

Cliente: python3 client.py -s <IP> -p <Puerto>

Batería de pruebas: Parte 1

ID	Operación	Propósito de la Prueba	Resultado Esperado
1	REGISTER user1	Registro inicial de un usuario válido en el sistema.	c> REGISTER OK
2	REGISTER user1	Control de errores: intento de registro con un nombre de usuario ya existente.	c> USERNAME IN USE (Retorno 1)
3	CONNECT user1	Conexión de un usuario válido. Verifica la creación del hilo demonio de escucha.	c> CONNECT OK
4	CONNECT user1	Control de errores: intento de conexión de un usuario que ya está conectado.	c> USER ALREADY CONNECTED (Retorno 2)
5	CONNECT user2	Control de errores: intento de conexión de un usuario no registrado.	c> CONNECT FAIL, USER DOES NOT EXIST (Retorno 1)
6	SEND user2 hola	Control de errores: enviar mensaje a un usuario no registrado en el servidor.	c> SEND FAIL, USER DOES NOT EXIST (Retorno 1)
7	REGISTER user2 SEND user2 hola	Envío asíncrono. user2 está registrado pero desconectado.	Servidor almacena el mensaje. Remitente ve: c> SEND OK MESSAGE 1.

8	CONNECT user2	Verificación de mensajes pendientes. Al conectarse, user2 debe recibir los mensajes cacheados.	user2 recibe: s> MESSAGE 1 FROM user1...
9	USERS (desde user1)	Comprobación de estado global.	Muestra (2 users connected) y la lista.
10	DISCONNECT user1	Cierre de sesión válido. El hilo de escucha se destruye localmente.	c> DISCONNECT OK
11	UNREGISTER user2	Borrado de cuenta. Purga de DB.	c> UNREGISTER OK

Batería de pruebas: Parte 2

ID	Operación	Propósito de la Prueba	Resultado Esperado
12	Web Service Test	Verificación de normalización. Enviar SEND user2 hola que tal.	El servidor recibe hola que tal. La consola RPC lo registra correctamente.
13	RPC Logging	Verificación de trazabilidad. Ejecutar REGISTER user3.	Consola RPC muestra: user3 REGISTER
14	SENDATTACH	Envío de mensaje con adjunto a usuario conectado. Prueba de parsing de parámetros.	Servidor devuelve ID. RPC muestra SENDATTACH /ruta/fichero.
15	SENDATTACH	Envío asíncrono con adjunto. Destinatario offline.	El servidor guarda en mensajes_usuario.dat tanto el ID como el nombre del fichero.
16	GETFILE	Transferencia P2P exitosa. user2 solicita el archivo a user1.	Socket directo transfiere bytes. Se guarda localFileName idéntico al original.
17	GETFILE (Offline)	Control de errores: user1 se desconecta antes de que user2 pida el fichero.	El cliente de user2 no logra contactar la IP/Puerto y muestra FILE TRANSFER FAILED....

Conclusiones

La realización de esta práctica nos ha permitido poner en práctica todos los conceptos de arquitecturas distribuidas estudiados durante el curso. Si bien ha sido un proyecto largo, ya que había que implementar muchas funcionalidades, muchas de ellas razonablemente complejas, sí que ha sido muy gratificante ver que todo funcionaba al final, y que podíamos mandarnos mensajes entre nosotros con algo que habíamos hecho nosotros mismos.

En definitiva, este proyecto nos ha gustado porque nos ha permitido entender cómo funciona algo que usábamos en nuestro día a día como son los servicios de mensajería.